LAST-DAY SELENIUM (PYTHON) REVISION SHEET

(L1–L2 Interview Ready)

1️⃣ CORE SETUP (MUST REMEMBER)

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
```

Launch:

```
driver = webdriver.Chrome()
driver.get(url)
driver.maximize_window()
```

Close:

```
driver.close()
driver.quit()
```

2️⃣ LOCATORS (ORDER OF PREFERENCE)

1. `ID`
2. `NAME`
3. `CSS`
4. `XPATH`

```
driver.find_element(By.ID, "id")
driver.find_elements(By.XPATH, "//a")
```

🔑 XPath basics:

```
//tag[@attr='value']
contains()
starts-with()
following-sibling::
```

3️⃣ WEB ELEMENT ACTIONS

```
element.click()
element.send_keys("text")
element.clear()
```

State checks:

```
element.is_displayed()
element.is_enabled()
element.is_selected()
```

4️⃣ WAITS (VERY IMPORTANT)

❌ Avoid:

```
time.sleep(5)
```

✅ Use:

```
driver.implicitly_wait(10)
```

✅ Preferred:

```
wait = WebDriverWait(driver, 10)
wait.until(EC.element_to_be_clickable((By.ID, "btn")))
```

Common conditions:

- `visibility_of_element_located`
- `presence_of_element_located`
- `element_to_be_clickable`
- `alert_is_present`

5️⃣ ALERTS / FRAMES / WINDOWS

Alert:

```
alert = driver.switch_to.alert
alert.accept()
```

Frame:

```
driver.switch_to.frame(index | name | element)
driver.switch_to.default_content()
```

Window:

```
parent = driver.current_window_handle
for h in driver.window_handles:
    if h != parent:
        driver.switch_to.window(h)
```

---

6 DROPDOWNS

```
dropdown = Select(element)
dropdown.select_by_visible_text("India")
```

---

7 RADIO BUTTONS & CHECKBOXES (DYNAMIC)

Radio:

```
for r in driver.find_elements(By.NAME, "gender"):
    if r.get_attribute("value") == "Male":
        r.click()
```

Checkbox:

```
for box in driver.find_elements(By.XPATH, "//label"):
    if box.text == "Selenium":
        box.click()
```

---

8 MOUSE & KEYBOARD ACTIONS

```
actions = ActionChains(driver)
actions.context_click(element).perform()    # right click
actions.double_click(element).perform()
actions.drag_and_drop(src, tgt).perform()
```

---

9 SCROLLING & JS CLICK

```
driver.execute_script("window.scrollBy(0,500)")
driver.execute_script("arguments[0].click();", element)
```

Use JS click when normal click fails.

---

10 FILE HANDLING WITH SELENIUM

Upload:

```
element.send_keys("C:/file.txt")
```

Download verification:

```
import os
os.path.exists("C:/Downloads/report.pdf")
```

Screenshot:

```
driver.save_screenshot("page.png")
```

1️⃣1️⃣ TABLE HANDLING

```
rows = driver.find_elements(By.XPATH, "//table//tr")
for row in rows:
    print(row.text)
```

Dynamic cell:

```
//td[text()='John']/following-sibling::td
```

1️⃣2️⃣ EXCEPTIONS (MENTION)

- `NoSuchElementException`
- `TimeoutException`
- `StaleElementReferenceException`

Handling stale:

Re-locate the element and retry.

1️⃣3️⃣ PAGE OBJECT MODEL (POM)

```
class LoginPage:
    def __init__(self, driver):
        self.driver = driver

    def login(self, user):
        self.driver.find_element(By.ID, "user").send_keys(user)
```

🔑 Interview line:

“POM improves readability and maintainability.”

1️⃣4️⃣ SELENIUM LIMITATIONS (ASKED OFTEN)

❌ Cannot handle:

- Captcha
- OS popups
- Mobile apps directly

1️⃣5️⃣ INTERVIEW ONE-LINERS (MEMORIZE)

- **Waits**: “I prefer explicit waits over sleep.”
- **Dynamic elements**: “I use relative XPath and waits.”
- **Frames/windows**: “Handled using `driver.switch_to`.”
- **Failures**: “Screenshots + logs for debugging.”
- **Design**: “Framework with POM and reusable methods.”

✅ PYTEST – QUICK REVISION NOTES (L1–L2)

1️⃣ WHAT IS PYTEST?

Pytest is a Python testing framework used to write, organize, and execute test cases with simple syntax.

Key points:

- No boilerplate
- Uses plain `assert`
- Supports fixtures, markers, parameterization

2️⃣ BASIC TEST STRUCTURE

Rules:

- File name → `test_*.py`
- Function name → `test_*`

```
def test_login():
    assert True
```

Run:

```
pytest
pytest -v
pytest -s
```

3️⃣ ASSERTIONS (VERY IMPORTANT)

```
assert a == b
assert a != b
assert a > b
assert "Login" in title
assert element.is_displayed()
```

With message:

```
assert status == 200, "API failed"
```

4️⃣ FIXTURES (MOST ASKED 🔥)

Definition (say this):

Fixtures are reusable setup and teardown functions used to prepare test environment or data.

Basic fixture

```
import pytest

@pytest.fixture
def setup():
    print("Setup")
    yield
    print("Teardown")
```

Usage:

```
def test_example(setup):
    assert True
```

Fixture scopes

```
@pytest.fixture(scope="function")
@pytest.fixture(scope="class")
@pytest.fixture(scope="module")
@pytest.fixture(scope="session")
```

One-liner:

Scope defines how often a fixture runs.

---

5 `conftest.py`

Purpose:

- Store **shared fixtures**
- No import required
- Auto-discovered by pytest

```
# conftest.py
@pytest.fixture
def driver():
    return webdriver.Chrome()
```

---

6 MARKERS

What is a marker?

Markers are used to tag and selectively run test cases.

```
import pytest

@pytest.mark.smoke
def test_login():
    assert True
```

Run:

```
pytest -m smoke
```

Register markers (`pytest.ini`):

```
[pytest]
markers =
    smoke: smoke tests
    regression: regression tests
```

---

7 PARAMETERIZATION (DATA-DRIVEN)

```
@pytest.mark.parametrize("a,b,result", [
    (1, 2, 3),
    (3, 4, 7)
])
def test_add(a, b, result):
    assert a + b == result
```

One-liner:

Parametrize allows running the same test with multiple data sets.

---

8 SKIP & XFAIL

Skip:

```
@pytest.mark.skip(reason="Not ready")
def test_skip():
    pass
```

Conditional skip:

```
@pytest.mark.skipif(sys.version_info < (3,8), reason="Old Python")
```

Xfail:

```
@pytest.mark.xfail(reason="Known bug")
def test_known_issue():
    assert False
```

Correct definition:

xfail means the test is expected to fail and won't mark the build as failed.

---

9 HOOKS (CLEAR + SIMPLE)

Definition:

Hooks are special functions used to customize pytest execution flow globally.

Example (mention-level):

```
def pytest_runtest_setup(item):
    print("Before test")
```

Difference:

- Fixture → setup/teardown
- Hook → modify pytest behavior

Safe interview line:

I mostly use fixtures; hooks are for framework-level customization.

🔟 RUNNING SPECIFIC TESTS

Specific file:

```
pytest test_login.py
```

Specific test:

```
pytest test_login.py::test_valid_login
```

Keyword:

```
pytest -k login
```

Markers:

```
pytest -m smoke
```

Include & exclude:

```
pytest -m "(smoke or regression) and not slow"
```

1️⃣1️⃣ PYTEST + SELENIUM (COMMON PATTERN)

```
@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    yield driver
    driver.quit()
```

Use:

```
def test_login(driver):
    driver.get("https://example.com")
```

1️⃣2️⃣ PYTEST.INI (CONFIG)

```
[pytest]
addopts = -v
testpaths = tests
markers =
    smoke
    regression
```

Purpose:

pytest.ini controls test discovery and execution behavior.

🧠 INTERVIEW ONE-LINERS (MEMORIZE)

- Fixtures → reusable setup/teardown
- Markers → test categorization
- Parametrize → data-driven testing
- Hooks → customize pytest execution
- conftest.py → shared fixtures

✅ FINAL 3-MIN CHECK

Ask yourself:

- Can I explain fixtures vs hooks?
- Can I explain markers?
- Can I run a single test?
- Can I explain xfail?

If yes → you're good.

1️⃣ FIXTURES (EXPANDED)

What it is

- Reusable setup & teardown mechanism
- Avoids code duplication

Why it is used

- Browser setup
- Test data creation
- DB / API connections
- Cleanup after test execution

Key concepts

- `@pytest.fixture`
- `yield` → teardown happens after test
- Fixtures are injected by name

Fixture scopes

- `function` – runs for every test
- `class` – once per class

- `module` – once per file
- `session` – once per run

Interview-ready line

“Fixtures are reusable setup and teardown functions with configurable scope.”

---

2 MARKERS (EXPANDED)

What it is? Tags applied to test cases

Why it is used

- Selective execution
- Smoke / regression / sanity separation
- CI/CD pipeline control

Types of markers

- Built-in: `skip`, `skipif`, `xfail`
- Custom: `smoke`, `regression`, `api`

How to run

```
pytest -m smoke
pytest -m "smoke or regression"
pytest -m "not slow"
```

Interview-ready line

“Markers are used to categorize tests and control selective execution.”

---

3 PARAMETERIZATION (EXPANDED)

What it is

- Running the same test with multiple data sets

Why it is used

- Data-driven testing
- Reduce duplicate test cases
- Improve coverage

Common data sources

- Inline tuples
- JSON files
- Excel sheets
- CSV files

Syntax

```
@pytest.mark.parametrize("a,b", [(1,2), (3,4)])
```

Interview-ready line

“Parameterization helps execute the same test with multiple inputs efficiently.”

4️⃣ HOOKS (EXPANDED – IMPORTANT FOR YOU)

What it is? Special functions that interact with pytest lifecycle

When hooks are used

- Logging
- Reporting
- Screenshot on failure
- Custom execution behavior

Common hooks (name recall only)

- `pytest_runtest_setup`
- `pytest_runtest_call`
- `pytest_runtest_teardown`
- `pytest_sessionstart`
- `pytest_sessionfinish`

Where they live: Usually inside `conftest.py`

Interview-ready line (SAFE): “Hooks customize pytest execution flow globally, whereas fixtures handle setup and teardown.”

5️⃣ TEST EXECUTION & CONFIGURATION (EXPANDED)

Ways to run tests

- Full suite
- Single file
- Single test
- By keyword
- By marker

Commands

```
pytest test_login.py
pytest test_login.py::test_valid_login
pytest -k login
pytest -m smoke
```

pytest.ini usage

- Default execution rules
- Marker registration
- Parallel execution config

Example

```
[pytest]
addopts = -v
markers =
    smoke
    regression
```

Interview-ready line: "pytest.ini is used to control test discovery and execution behavior."

🧠 FINAL MEMORY ANCHOR (WRITE THIS ON TOP OF PAGE)

Fixtures → setup & teardown
Markers → test selection
Parametrize → data-driven tests
Hooks → pytest lifecycle customization
pytest.ini → execution control
